Logistic Regression - Python Source Code

```
"""
Logistic Regression: Diabetes Prediction
Run from VS Code terminal:
    python logistic_regression.py
"""
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_auc_score, roc_curve
)

BASE_DIR = Path(__file__).resolve().parent
DATA_PATH = BASE_DIR / "data" / "diabetes.csv"
OUTPUT_DIR = BASE_DIR / "outputs"
OUTPUT_DIR.mkdir(exist_ok=True)

FEATURES = [
    "Age", "BMI", "BloodPressure", "Insulin",
    "Glucose", "DiabetesPedigreeFunction"
]
TARGET = "Outcome"


def main():
    if not DATA_PATH.exists():
        raise FileNotFoundError(
            f"Dataset not found: {DATA_PATH}\n"
            "Keep diabetes.csv inside the project's data folder."
        )

    df = pd.read_csv(DATA_PATH)
    required = FEATURES + [TARGET]
    missing = [column for column in required if column not in df.columns]
    if missing:
        raise ValueError(f"Dataset is missing columns: {missing}")

    print("DATASET PREVIEW")
    print(df.head())
    print(f"\nDataset shape: {df.shape}")
    print("\nTarget distribution:")
    print(df[TARGET].value_counts().sort_index())

    X = df[FEATURES]
    y = df[TARGET]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    model = Pipeline([
        ("scaler", StandardScaler()),
        ("logistic_regression", LogisticRegression(max_iter=1000))
    ])
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    y_probability = model.predict_proba(X_test)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    auc_score = roc_auc_score(y_test, y_probability)
    cm = confusion_matrix(y_test, y_pred)

    print("\nMODEL RESULTS")
    print(f"Training rows: {len(X_train)}")
    print(f"Testing rows:  {len(X_test)}")
    print(f"Accuracy:      {accuracy:.4f} ({accuracy * 100:.2f}%)")
    print(f"ROC-AUC:       {auc_score:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=["No Diabetes", "Diabetes"]))

    print("Confusion Matrix:")
    print(cm)

    # Save confusion matrix plot.
    plt.figure(figsize=(6, 5))
    sns.heatmap(
        cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=["No Diabetes", "Diabetes"],
        yticklabels=["No Diabetes", "Diabetes"]
    )
    plt.title("Logistic Regression - Confusion Matrix")
```

```
    plt.xlabel("Predicted label")
    plt.ylabel("Actual label")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "confusion_matrix.png", dpi=150)
    plt.close()

    # Save ROC curve.
    fpr, tpr, _ = roc_curve(y_test, y_probability)
    plt.figure(figsize=(6, 5))
    plt.plot(fpr, tpr, label=f"ROC-AUC = {auc_score:.3f}")
    plt.plot([0, 1], [0, 1], linestyle="--", label="Random classifier")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve")
    plt.legend()
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "roc_curve.png", dpi=150)
    plt.close()

    # Export model coefficients in the standardized feature space.
    classifier = model.named_steps["logistic_regression"]
    coefficients = pd.DataFrame({
        "Feature": FEATURES,
        "Coefficient": classifier.coef_[0]
    }).sort_values("Coefficient", ascending=False)
    coefficients.to_csv(OUTPUT_DIR / "model_coefficients.csv", index=False)

    print(f"\nFiles saved in: {OUTPUT_DIR}")
    print("- confusion_matrix.png")
    print("- roc_curve.png")
    print("- model_coefficients.csv")


if __name__ == "__main__":
    main()
```